Slide 1: Matrix Partitioning & Load Balancing

Speaker: Fuzail

The Problem

Compute **C = A × B** on a distributed memory system where matrices may be too large to fit in a single node's RAM.

N × N **A** × N × N **B** = N × N **C**

Approach : 1D Row Wise Decomposition

SPMD Approach

Single Program, Multiple Data. Each rank executes the same multiplication logic on its assigned row slice.

Load Balancing

If `n % size != 0`, rows cannot be evenly divided. Some ranks will naturally be assigned more work than others.

My Solution

Base rows: `n / size`

The first `remainder` processes receive exactly 1 extra row.

```
rows_per_proc = n / size + (i < remainder ? 1 : 0);
```

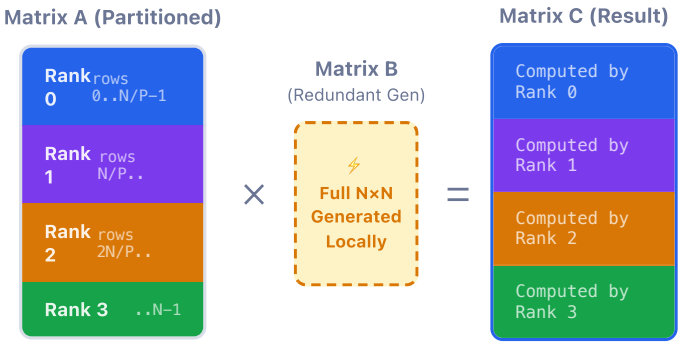
Why This Is Balanced

- **Negligible Imbalance:** Maximum difference between any two processes is at most 1 row. For large `n` (e.g., 8000), this is mathematically insignificant.
- **No Idle Time:** Ensures an incredibly even workload distribution.
- **Good Scalability:** Prevents faster nodes from idling at the final implicit barrier.

Partitioning Visualization (Zero-Comm)

Ranks: P=4

Matrix A is split row-wise across ranks. Matrix B is **fully generated locally on every rank** using redundant CPU cycles.



matmul.c — row distribution & displacement calculation

```c
for (int i = 0; i < size; i++) {
    int rows_per_proc = n / size + (i < remainder ? 1 : 0);
    sendcounts[i] = rows_per_proc * n;
    displs[i] = sum;

    if (i == rank) {
        my_rows = rows_per_proc;
        my_row_start = sum / n;  // Used to offset PRNG seed
    }
    sum += sendcounts[i];
}
```